Web Development I

CS 212

គោលបំណង

ក្នុងមេរៀននេះយើងនឹងសិក្សាទៅលើ៖

- ការស្វែងអំពី Pattern ដែលគេនិយមប្រើប្រាស់ដើម្បីបង្កើតវេបសាយ
- ការស្វែងយល់អំពី back-end និង front-end
- ការរៀបចំ Files management
- Concept ជាមូលដ្ឋានដើម្បីបង្កើតនូវ Web Application
- ការបង្កើតនូវ API (Application Programming Interface)

មាតិកា | CONTENTS

ជំពូកទី១. កិច្ចចាប់ផ្តើម

1. ការណែនាំអំពី MVC pattern design
2. ការណែនាំអំពី Django MVT
3. Why Django ?

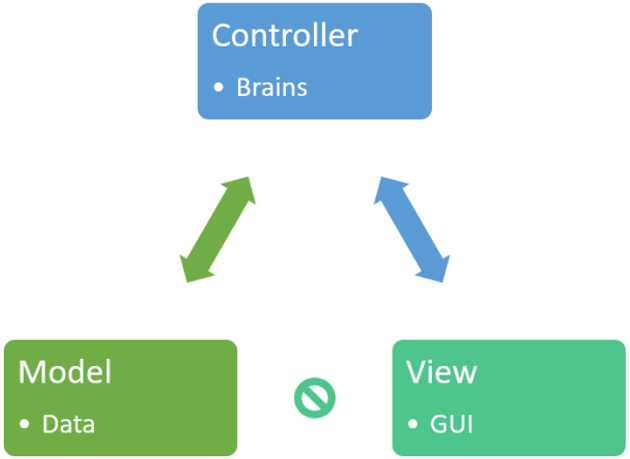
១ការណែនាំអំពី MVC pattern design

- MVC មកពី Model-View-Controller
- MVC ត្រូវបានគេបង្កើតឡើងដើម្បីធ្វើការដោះនិងគ្រប់គ្រងនូវការបង្កើតនូវ application ខុសៗគ្នាទៅតាមការចង់បាន។ ម្យ៉ាងវិញទៀតវាត្រូវបានប្រើសម្រាប់ញែកនិងបង្ហាញពី business logic។
- វាត្រូវបានគេនិយមប្រើប្រាស់បំផុតនៅក្នុង App និង Web Development
- MVC មានសមាសធាតុបីសំខាន់ៗចូលរួម៖

1. M: Model ប្រើសំរាប់រក្សានិងរៀបចំទិន្នន័យ។ ជាទូទៅវាមានទំនាក់ទំនងទៅនឹង database
2. V: Views ទាក់ទងទៅនឹង Graphical users interface។ វាមានតួនាទីបង្ហាញពីទិន្នន័យដូចជា Chart, diagram, តារាង , form ជាដើម។ វាផ្ទុកនូវរាល់ functionality ដែលមានអំពើជាមួយនឹងអ្នកប្រើប្រាស់
3. C: Controller: ប្រៀបដូចទៅនឹងខួរក្បាលនៃ Application។ controller ជាអ្នកភ្ជាប់រវាង model និង view។ controller មានតួនាទីសំខាន់សម្រាប់រៀបចំ data រវាង view និង controller។

១ ការណែនាំអំពី MVC pattern design

១.១ ស្វែងយល់អំពីរចនាសម្ព័ន្ធនៃ MVC



Example: About MVC

១ ការណែនាំអំពី MVC pattern design?

១.២ អត្ថប្រយោជន៍

-ជាទូទៅយើងប្រើប្រាស់វាក្នុង GUIs

-ពេញនិយមសម្រាប់វេប applications

-វាបែងចែកយ៉ាងច្បាស់លាស់រវាង client & server side នៃវេប

-ជួយក្នុងការ design pattern នៅពេលរៀបគម្រោងអភិវឌ្ឍន៍នូវ project

-កូដត្រូវបានបែងចែកយ៉ាងច្បាស់លាស់រវាង Model, View និង Controller

-ជួយលុបចេញនូវ dependencies ដែលមិនចាំបាច់

-អាចប្រើប្រាស់ឡើងវិញដោយមិនចាំបាច់កែប្រែ

-Code reuse

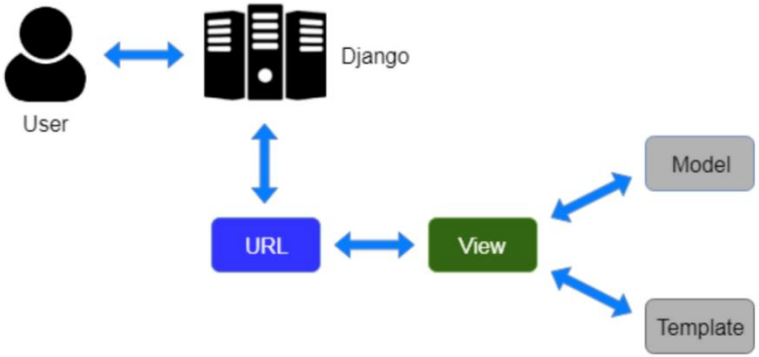
-ងាយស្រួលគ្រប់គ្រងនិងកែប្រែ

-Supports នូវ views ជាច្រើ

២ ការណែនាំអំពី Django MVT?

- MVT- Model-View-Template គឺជា Software design pattern មួយប្រភេទ។
- Model ជួយធ្វើការ handle database។ វាគឺជាស្រទាប់នៃ data access។
- Template គឺជាអ្នកបង្ហាញពី user interface part។
- View គឺត្រូវបានប្រើប្រាស់ដើម្បីប្រតិបត្តិការទៅលើផ្នែក business logic និងមានទំនាក់ទំនងជាមួយនឹង model មួយដើម្បីពាំនាំទិន្នន័យហើយ render នូវ template មួយផងដែរ។

- Although Django follows MVC pattern but maintains it?s own conventions. So, control is handled by the framework itself.
- There is no separate controller and complete application is based on Model View and Template. That?s why it is called MVT application.



Example:

-A user **requests** for a resource to the Django, Django works as a controller and check to the available resource in URL.

-If URL maps, **a view is called** that interact with model and template, it renders a template.

-Django responds back to the user and sends a template as a **response**.

២.១ Model

- In Django, a model is a class which is used to contain essential fields and methods. Each model class maps to a single table in the database.
- Django Model is a subclass of **django.db.models.Model** and each field of the model class represents a database field (column).
- Django provides us a database-abstraction API which allows us to create, retrieve, update and delete a record from the mapped table.

Example:

```
from django.db import models

class Employee(models.Model):

    first_name = models.CharField(max_length=30)

    last_name = models.CharField(max_length=30)
```

២.២ View

- A view is a place where we put our business logic of the application. The view is a python function which is used to perform some business logic and return a response to the user. This response can be the HTML contents of a Web page, or a redirect, or a 404 error.
- All the view function are created inside the **views.py** file of the Django app.

Example:

//views.py

```
import datetime
# Create your views here.
from django.http import HttpResponse
def index(request):
    now = datetime.datetime.now()
    html = "<html><body><h3>Now time is %s.</h3></body></html>" % now
    return HttpResponse(html)    # rendering the template in HttpResponse
```

២.៣ Template

- Django provides a convenient way to generate dynamic HTML pages by using its template system.
- A template consists of static parts of the desired HTML output as well as some special syntax describing how dynamic content will be inserted.

Why Django Template?

- In HTML file, we can't write python code because the code is only interpreted by python interpreter not the browser. We know that HTML is a static markup language, while Python is a dynamic programming language.
- Django template engine is used to separate the design from the python code and allows us to build dynamic web pages.

Example:

// index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Index</title>
</head>
<body>
<h2>Welcome to Django!!!</h2>
</body>
</html>
```

Django Template Configuration

To configure the template system, we have to provide some entries in **settings.py** file.

```python
TEMPLATES = [
  {
    'BACKEND': 'django.template.backends.django.DjangoTemplates',
    'DIRS': [os.path.join(BASE_DIR,'templates')],
    'APP_DIRS': True,
    'OPTIONS': {
      'context_processors': [
        'django.template.context_processors.debug',
        'django.template.context_processors.request',
        'django.contrib.auth.context_processors.auth',
        'django.contrib.messages.context_processors.messages',
      ],
    },
  },
]
```

៣ Why Django?

•Django គឺជា High-leve python web framework មួយដែលអាចធ្វើការអភិវឌ្ឍន៍ web application បានឆាប់រហ័ស និងងាយស្រួលធ្វើការគ្រប់គ្រង

•វាជួយយើងដូចជា៖

- Complete វាផ្តល់នូវរាល់អ្វីៗដែល developer ចង់បាន។

-Versatile ប្រើប្រាស់សំរាប់បង្កើតនូវ website គ្រប់ប្រភេទ។ វាអាចធ្វើការចែកចាយនូវ HTML, RSS feeds, JSON, XML, etc

-Secure វាជួយ developers ជៀសវាងនូវកំហុស security ទូទៅដោយផ្តល់នូវ freamework មួយដើម្បីធ្វើនូវរឿងដែលត្រឹមត្រូវ។ វាផ្តល់នូវវិធីសាស្ត្រសុវត្ថិភាពដើម្បីគ្រប់គ្រងនូវ user accounts និង លេខសំងាត់ ហើយជៀសវាងនូវកំហុសជាទូទៅដូចជាការដាក់នូវ seesion information ក្នុង cookies ដោយមិនបានប្រើប្រាស់ password hash

- Scalable វាអាចធ្វើការបង្កើននូវ traffic ដោយបន្ថែមនូវ hardware នៅគ្រប់ level: caching servers, database servers, រឺ application server។

- Maintainable: Django code is written using design principles and patterns that encourage the creation of maintainable and reusable code (Don't Repeat Yourself (DRY))

- Portable Django is written in Python, which runs on many platforms. That means that you are not tied to any particular server platform, and can run your applications on many flavours of Linux, Windows, and Mac OSX.

Homework

Thank You !